Table des matières

Schéma de la base de données - moyun-pos 1
1. Tables de référence . . . 1
2. Tables transactionnelles . . . 3
3. Tables fiscales (append-only) . . . 4
4. Tables Peppol (build cert) . . . 5
5. Tables d'intégration webshop (inutilisées dans le build cert) . . . 6
6. Migrations . . . 6
7. Méthode d'export pour audit . . . 6

Schéma de la base de données - moyun-pos

Conformément au §II.C du Manuel de certification SCE 2.0, ce document liste chaque table, chaque champ, son type, sa longueur (le cas échéant) et sa description.

Moteur : SQLite 3.x bundled via `rusqlite` (Rust). Encodage : UTF-8. Mode : WAL (Write-Ahead Logging) - atomique crash-safe. Fichier vivant : `/var/lib/moyun-pos/pos.db` en production. Source de vérité du schéma : `crates/pos-db/src/schema.rs` (lecture machine via `sqlite3 pos.db ".schema"`).

1. Tables de référence

1.1 `languages`

Langues prises en charge par les traductions multi-langues.

Champ	Type	Longueur	Description
`code`	TEXT	2	Clé primaire. Code ISO 639-1 : `fr`, `nl`, `en`, `zh`
`name`	TEXT	-	Nom de la langue dans cette langue (“Français”, “中文”…)
`active`	INTEGER	1 (booléen)	1 = langue activée pour ce déploiement, 0 = désactivée

Données initiales (insérées par la migration `001` - voir `crates/pos-db/src/migrate.rs`) : `fr`, `nl`, `en`, `zh`.

1.2 `translations`

Table polymorphe : associe une chaîne traduite à n'importe quelle entité de la base (catégorie, item, modificateur…).

Champ	Type	Description
`id`	INTEGER	Clé primaire auto-incrémentée
`entity_type`	TEXT	Type d'entité : `category`, `menu_item`, `modifier_group`, `modifier`
`entity_id`	INTEGER	ID de l'entité concernée
`field`	TEXT	Nom du champ traduit : typiquement `name`, parfois `description`
`lang`	TEXT (FK → `languages.code`)	Code langue de la traduction

Champ	Type	Description
value	TEXT	Texte traduit

Contrainte d'unicité : UNIQUE(entity_type, entity_id, field, lang). Index : idx_translations_lookup(enti
entity_id, lang).

1.3 **categories**

Catégories de menu (ex. "Sushi", "Boissons").

Champ	Type	Description
id	INTEGER	Clé primaire
external_id	TEXT (unique)	Référence externe (NULL en build cert)
sort_order	INTEGER	Ordre d'affichage
color	TEXT	Couleur hexadécimale optionnelle
active	INTEGER	Drapeau actif (0/1)

1.4 **menu_items**

Articles vendus. Chaque article appartient à exactement une catégorie et porte UN code TVA SCE 2.0 (A/B/C/D/X).

Champ	Type	Description
id	INTEGER	Clé primaire
external_id	TEXT (unique)	Référence externe (NULL en build cert)
category_id	INTEGER (FK → categories.id)	Catégorie parente
price_cents	INTEGER	Prix unitaire TVA incluse en centimes d'euro
vat_code	TEXT	Code TVA SCE : A, B, C, D, X
vat_category	TEXT	Catégorie fiscale webshop : food, beverage, zero_rated, out_of_scope
kitchen_printer	INTEGER	1 = à envoyer en cuisine, 0 = imprimante caisse seulement
sort_order	INTEGER	Ordre d'affichage dans la grille
active	INTEGER	1 = article disponible, 0 = retiré du menu

1.5 **modifier_groups**

Groupes de modificateurs liés à un article (ex. "Sauce" pour les sushi).

Champ	Type	Description
id	INTEGER	Clé primaire
external_id	TEXT (unique)	Référence externe
min_select	INTEGER	Nombre minimum de modificateurs à sélectionner
max_select	INTEGER	Nombre maximum

Champ	Type	Description
menu_item_id	INTEGER (FK → menu_items.id)	Article parent

1.6 **modifiers**

Options individuelles d'un groupe (ex. "Sauce soja").

Champ	Type	Description
id	INTEGER	Clé primaire
external_id	TEXT (unique)	Référence externe
group_id	INTEGER (FK → modifier_groups.id)	Groupe parent
price_cents	INTEGER	Supplément de prix (peut être 0)
sort_order	INTEGER	Ordre dans le groupe

2. Tables transactionnelles

2.1 **orders**

Commandes (dine-in et takeaway). Une commande peut être ouverte (open), fermée (closed - encaissée) ou annulée (voided).

Champ	Type	Description
id	INTEGER	Clé primaire
table_number	INTEGER (nullable)	Numéro de table pour les dine-in
order_type	TEXT	dine_in ou takeaway
status	TEXT	open, closed ou voided
user_insz	TEXT	INSZ de l'opérateur qui a ouvert la commande
created_at	TEXT (ISO 8601)	Date / heure d'ouverture
closed_at	TEXT (nullable)	Date / heure d'encaissement
subtotal_cents	INTEGER	Sous-total TVA incluse
total_cents	INTEGER	Total payé en centimes
hubrise_order_id	TEXT (nullable)	ID commande externe (NULL dans le build cert)
is_fiscal	INTEGER	1 = dine-in soumis au FDM, 0 = webshop hors FDM (irrelevant en cert)
source	TEXT	pos_local ou webshop
hubrise_status	TEXT (nullable)	Statut webshop
service_type	TEXT (nullable)	Type de service webshop
raw_hubrise_json	TEXT (nullable)	Payload commande externe brut

2.2 **order_lines**

Lignes d'une commande. Dénormalisées (le nom et le prix sont copiés depuis menu_items au moment de l'ajout, pour figer le tarif historique).

Champ	Type	Description
id	INTEGER	Clé primaire
order_id	INTEGER (FK → orders.id)	Commande parente
menu_item_id	INTEGER (FK → menu_items.id)	Article référencé

Champ	Type	Description
quantity	INTEGER	Quantité (peut être négative pour les remboursements)
unit_price_cents	INTEGER	Prix unitaire au moment de l'ajout (figé)
vat_code	TEXT	Code TVA appliqué (figé)
name	TEXT	Désignation imprimée (figée)
modifiers_json	TEXT (nullable)	Modificateurs sélectionnés en JSON
created_at	TEXT	Date / heure d'ajout

2.3 **payments**

Paiements enregistrés pour une commande. Plusieurs paiements possibles par commande (espèces + carte sur un partage). Modes limités à `cash`, `card`, `other` (extensible - vouchers).

Champ	Type	Description
id	INTEGER	Clé primaire
order_id	INTEGER (FK → orders.id)	Commande payée
method	TEXT	cash, card ou other
amount_cents	INTEGER	Montant en centimes (positif)
created_at	TEXT	Date / heure du paiement

3. Tables fiscales (append-only)

3.1 **fiscal_events** Attention : APPEND-ONLY

Le cœur du système fiscal. Une ligne par événement SCE 2.0 envoyé au FDM, jamais modifiée ni supprimée après insertion.

Champ	Type	Description
id	INTEGER	Clé primaire auto-incrémentée
event_type	TEXT	N, P, T, C, I, F, S, ou R (contraint par CHECK)
sequence_number	INTEGER	Compteur local POS (cf. COUNTER_STRATEGY.md)
order_id	INTEGER (FK → orders.id, nullable)	Commande associée pour N/P/I/C ; NULL pour F/S/R
pos_id	TEXT	Identifiant POS 14 caractères (cf. spec POS v2.2)
terminal_id	TEXT	ID du terminal physique
user_insz	TEXT	INSZ de l'opérateur
event_data_json	TEXT	Payload SCE complet sérialisé en JSON
fdm_response_json	TEXT (nullable)	Réponse brute du FDM (signature, counters, footer, QR)
fdm_id	TEXT (nullable)	ID du FDM extrait de la réponse
fdm_datetime	TEXT (nullable)	fdmDateTime extrait
event_label	TEXT (nullable)	Label retourné par le FDM (peut différer de event_type en formation)
short_signature	TEXT (nullable)	10 caractères hexa pour le ticket
qr_data	TEXT (nullable)	URL 38 caractères pour le QR code

Champ	Type	Description
`created_at`	TEXT	Date / heure d'insertion (= clôture de la tx)
`archived`	INTEGER	Vestige de schéma - toujours 0, candidat à suppression

Contrainte d'unicité : `UNIQUE(event_type, sequence_number)` - garantit la stricte séquentialité par type d'événement.

3.2 **`fiscal_counters`**

Compteurs persistants par label d'événement.

Champ	Type	Description
`event_label`	TEXT	Clé primaire = label SCE (`N`, `P`, `T`, `C`, `I`, `F`, `S`, `R`)
`current_value`	INTEGER	Dernier compteur émis pour ce label

Modification exclusive via `pos_fiscal::counter::FiscalCounter::next` (atomique avec l'archive, cf. `COUNTER_STRATEGY.md`).

3.3 **`z_reports`**

Index des rapports Z générés. Référence le `fiscal_events` correspondant.

Champ	Type	Description
`id`	INTEGER	Clé primaire
`report_date`	TEXT	Date de la journée fiscale clôturée
`sequence_number`	INTEGER	Numéro de Z séquentiel
`data_json`	TEXT	Snapshot des agrégats au moment du Z
`fiscal_event_id`	INTEGER (FK → `fiscal_events.id`)	Lien vers l'événement R signé
`created_at`	TEXT	Date / heure de génération

3.4 **`users`**

Utilisateurs du POS. INSZ comme clé primaire. Voir `PROFILS_UTILISATEURS_FR.md`.

Champ	Type	Description
`insz`	TEXT (11 chiffres)	Clé primaire. Numéro National Belge
`name`	TEXT	Nom affichable
`role`	TEXT	`admin`, `cashier`, ou `waiter` (contraint)
`pin_hash`	TEXT (nullable)	Hash bcrypt du PIN optionnel
`active`	INTEGER	1 = utilisateur actif, 0 = désactivé

4. Tables Peppol (build cert)

4.1 **`peppol_deliveries`**

Audit trail des factures B2B livrées via Access Point Peppol.

Champ	Type	Description
id	INTEGER	Clé primaire
fiscal_event_id	INTEGER (FK)	Lien vers l'événement I correspondant
invoice_no	TEXT	Numéro de facture émetteur
customer_endpoint_id	TEXT	Identifiant Peppol du destinataire (0208:BE...)
ap_vendor	TEXT	Nom du fournisseur AP ("storecove", "unifiedpost", "logging-only")
ap_message_id	TEXT (nullable)	ID retourné par l'AP
ubl_xml	TEXT	Document UBL 2.1 complet pour audit
ap_response_json	TEXT (nullable)	Réponse de l'AP
status	TEXT	queued, accepted, ou failed (contraint)
error_msg	TEXT (nullable)	Message d'erreur si failed
created_at	TEXT	Date / heure de l'envoi
accepted_at	TEXT (nullable)	Date / heure d'acceptation par l'AP

Index : idx_peppol_deliveries_invoice(invoice_no).

5. Tables d'intégration webshop (inutilisées dans le build cert)

Les tables hubrise_connection, hubrise_events, hubrise_outbox existent dans le schéma mais sont inutilisées en build cert —aucun code du binaire cert ne lit ni écrit ces tables (les intégrations tierces sont exclues à la compilation via feature Cargo, cf. BASELINE_TEST_REPORT.md). Elles sont préservées pour permettre qu'un déploiement non-cert lise une base créée par un build cert et inversement, sans migration de schéma.

6. Migrations

Système de migrations versionné dans crates/pos-db/src/migrate.rs. À la version actuelle, une seule migration existe (001 - initial-schema) qui crée toutes les tables ci-dessus. Les migrations sont consignées dans la table _migrations_applied (version, name, applied_at).

Les bases "legacy" créées avant la versification sont automatiquement annotées comme v1 (back-filled).

7. Méthode d'export pour audit

Voir FEUILLE_ACQUISITION_DONNEES_FR.md §2.